



# 웹 시스템 개발 1

클로저

# 클로저

- 자신을 내포하는 함수의 컨텍스트에 접근할 수 있는 함수
- 함수가 특정 스코프에 접근할 수 있도록 그 스코프에서 정의하는 것
- 함수를 선언할 때 만들어지는 유효범위가 사라진 후에도 호출할 수 있는 함수
- 이미 생명 주기가 끝난 외부 함수의 변수를 참조하는 함수
- 자유 변수가 있는 함수와 자유 변수를 알 수 있는 환경의 결합
- 로컬 변수를 참조하고 있는 함수 내의 함수
- 자신이 생성될 때의 스코프에서 알 수 있었던 변수들 중 언젠가 자신이 실행될 때 사용할 변수들만을 기억하여 유지시키는 함수

# 클로저

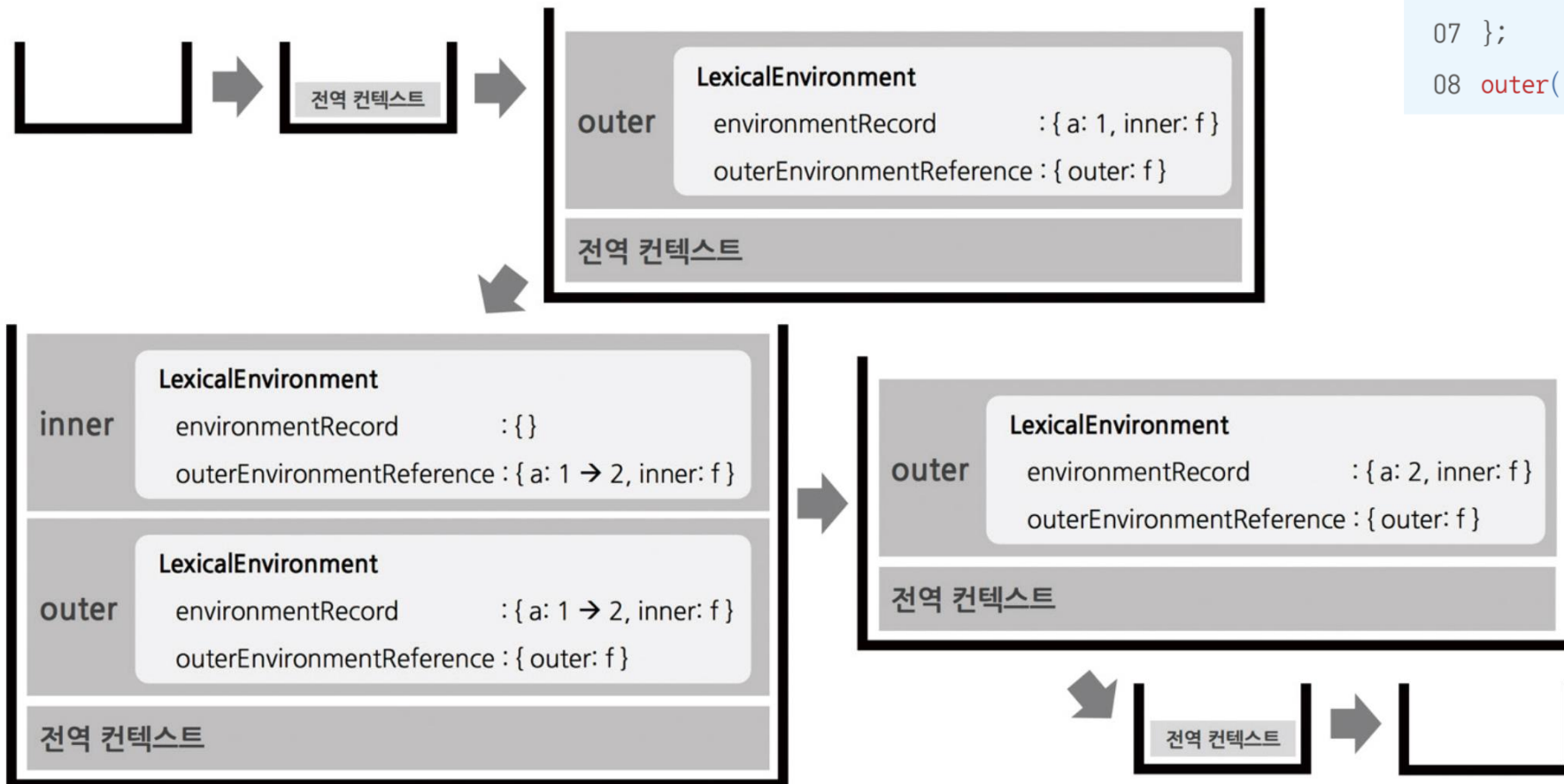
예제 5-1 외부 함수의 변수를 참조하는 내부 함수(1)

```
01 var outer = function () {  
02     var a = 1;  
03     var inner = function () {  
04         console.log(++a);  
05     };  
06     inner();  
07 };  
08 outer();
```

- ▶ A closure is the combination of a function and the lexical environment within which that function was declared.
- ▶ 어떤 컨텍스트 A에서 선언한 내부 함수 B의 실행 컨텍스트가 활성화된 시점에는 B의 `outerEnvironmentReference`가 참조하는 대상인 A의 LexicalEnvironment에도 접근 가능.
  - ▶ A에서는 B에서 선언한 변수에 접근할 수 없지만, B에서는 A에서 선언한 변수에 접근 가능

# 클로저

그림 5-1 일반적인 상황에서의 콜스택 흐름

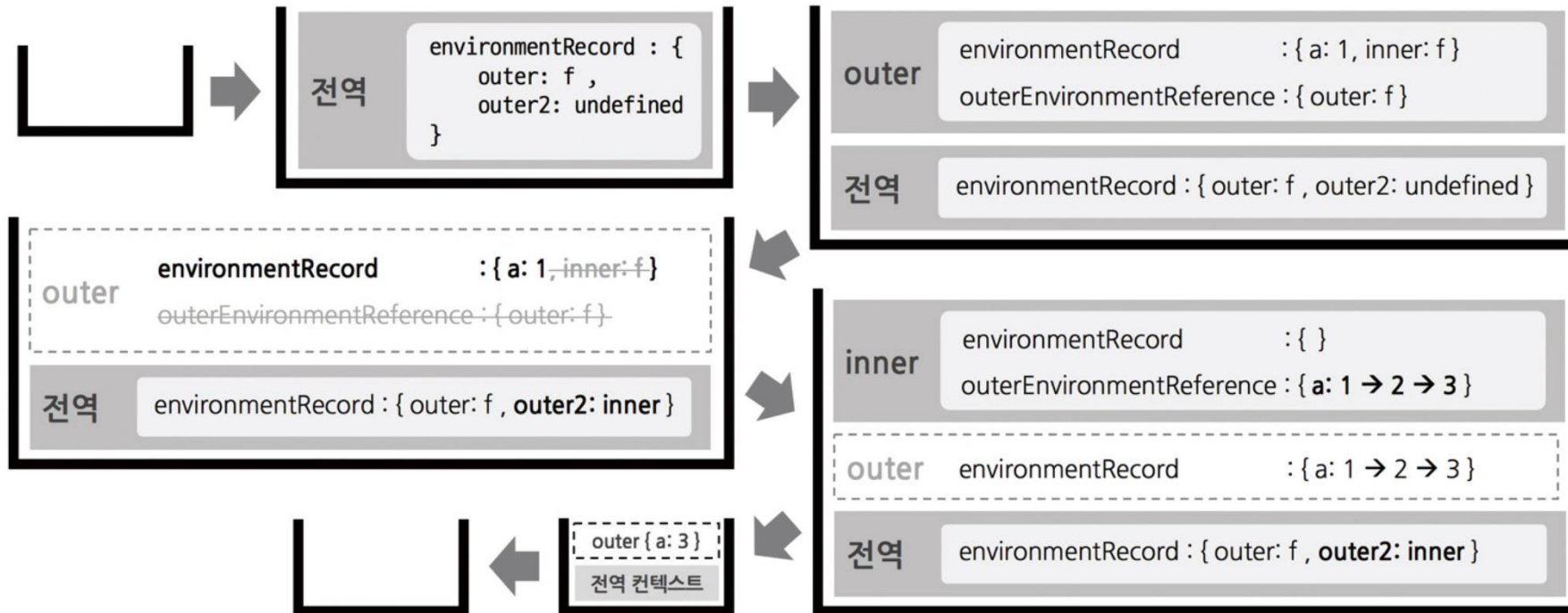


예제 5-1 외부 함수의 변수를 참조하는 내부 함수(1)

```
01 var outer = function () {  
02     var a = 1;  
03     var inner = function () {  
04         console.log(++a);  
05     };  
06     inner();  
07 };  
08 outer();
```

# 클로저

그림 5-2 클로저 발생 시의 콜스택 흐름



예제 5-3 외부 함수의 변수를 참조하는 내부 함수(3)

```
01 var outer = function () {  
02     var a = 1;  
03     var inner = function () {  
04         return ++a;  
05     };  
06     return inner;  
07 };  
08 var outer2 = outer();  
09 console.log(outer2()); // 2  
10 console.log(outer2()); // 3
```

# 클로저

- ▶ inner 함수의 실행 시점에는 outer 함수는 이미 실행 종료 상태인데 outer 함수의 LexicalEnvironment에 접근할 수 있는 이유는?
  - ▶ 가비지 컬렉터의 동작 방식 때문
- ▶ 가비지 컬렉터는 어떤 값을 참조하는 변수가 하나라도 있다면 그 값은 수집 대상에 포함시키지 않음
  - ▶ outer의 LexicalEnvironment에 속하는 변수 a가 가비지 컬렉터의 수집 대상에서 제외됨.
- ▶ 클로저의 정의
  - ▶ 클로저란 어떤 함수 A에서 선언한 변수 a를 참조하는 내부함수 B를 외부로 전달할 경우 A의 실행 컨텍스트가 종료된 이후에도 변수 a가 사라지지 않는 현상

예제 5-3 외부 함수의 변수를 참조하는 내부 함수(3)

```
01 var outer = function () {  
02     var a = 1;  
03     var inner = function () {  
04         return ++a;  
05     };  
06     return inner;  
07 };  
08 var outer2 = outer();  
09 console.log(outer2());    // 2  
10 console.log(outer2());    // 3
```



# 클로저

예제 5-3 외부 함수의 변수를 참조하는 내부 함수(3)

```
01 var outer = function () {  
02     var a = 1;  
03     var inner = function () {  
04         return ++a;  
05     };  
06     return inner;  
07 };  
08 var outer2 = outer();  
09 console.log(outer2());    // 2  
10 console.log(outer2());    // 3
```

- ▶ 자신을 내포하는 함수의 컨텍스트에 접근할 수 있는 함수
- ▶ 함수가 특정 스코프에 접근할 수 있도록 그 스코프에서 정의하는 것
- ▶ 함수를 선언할 때 만들어지는 유효범위가 사라진 후에도 호출할 수 있는 함수
- ▶ 이미 생명 주기상 끝난 외부 함수의 변수를 참조하는 함수
- ▶ 자유 변수가 있는 함수와 자유 변수를 알 수 있는 환경의 결합
- ▶ 로컬 변수를 참조하고 있는 함수 내의 함수
- ▶ 자신이 생성될 때의 스코프에서 알 수 있었던 변수들 중 언젠가 자신이 실행될 때 사용할 변수들만을 기억하여 유지시키는 함수

# 클로저

- return 없이도 클로저가 발생하는 경우

```
1
2 (function() {
3     var a = 0;
4     var intervalId = null;
5     var inner = function() {
6         if (++a >= 10) {
7             clearInterval(intervalId);
8         }
9         console.log(a);
10    };
11    intervalId = setInterval(inner, 1000);
12   })();
13
```

```
1
2 (function() {
3     var count = 0;
4     var button = document.createElement('button');
5     button.innerText = 'click';
6     button.addEventListener('click', function() {
7         console.log(++count, 'times clicked');
8     });
9     document.body.appendChild(button);
10   })();
11
```



# 클로저와 메모리 관리

- 클로저는 메모리 누수의 위험성을 가지고 있음
- 클로저는 어떤 필요에 의해 의도적으로 함수의 지역 변수가 메모리를 소모하도록 함으로써 발생
- 관리 방법
  - 식별자에 참조형이 아닌 기본형 데이터 (보통 null or undefined)를 할당
    - 참조 카운트를 0으로 만들면, 언젠가 GC가 수거

# 클로저와 메모리 관리

```
1
2  var outer = (function () {
3      var a = 1;
4      var inner = function () {
5          return ++a;
6      };
7      return inner;
8  })();
9  console.log(outer());
10 console.log(outer());
11 outer = null;
12
```

```
1
2  (function() {
3      var a = 0;
4      var intervalId = null;
5      var inner = function() {
6          if (++a >= 10) {
7              clearInterval(intervalId);
8              inner = null; // inner 식별자의 함수 참조를 끊음
9          }
10         console.log(a);
11     };
12     intervalId = setInterval(inner, 1000);
13 })();
14
```

# 클로저 활용 사례

- ▶ 콜백 함수 내부에서 외부 데이터를 사용하고자 할 때

addEventListener 함수의 콜백 함수가  
콜백 함수 외에도 사용된다면?

```
1
2  var fruits = ['apple', 'banana', 'peach'];
3  var $ul = document.createElement('ul');
4
5  fruits.forEach(function (fruit) {
6      var $li = document.createElement('li');
7      $li.innerText = fruit;
8      $li.addEventListener('click', function () {
9          alert('your choice is ' + fruit);
10     });
11     $ul.appendChild($li);
12 });
13
14 document.body.appendChild($ul);
15
```

반복을 줄이기 위해 외부로 분리,  
인자의 순서가 변경되고, this가 원래의 것과 달라짐

```
1
2  var fruits = ['apple', 'banana', 'peach'];
3  var $ul = document.createElement('ul');
4
5  var alertFruit = function (fruit) {
6      alert('your choice is ' + fruit);
7  };
8
9  fruits.forEach(function (fruit) {
10     var $li = document.createElement('li');
11     $li.innerText = fruit;
12     $li.addEventListener('click', alertFruit.bind(null, fruit));
13     $ul.appendChild($li);
14 });
15
16 document.body.appendChild($ul);
17
```

## 클로저 활용 사례

- ▶ 콜백 함수 내부에서 외부 데이터를 사용하고자 할 때

```
1
2  var fruits = ['apple', 'banana', 'peach'];
3  var $ul = document.createElement('ul');
4
5  var alertFruitBuilder = function (fruit) {
6    |   return function () {
7    |       alert('your choice is ' + fruit);
8    |   };
9  };
10
11  fruits.forEach(function (fruit) {
12    |   var $li = document.createElement('li');
13    |   $li.innerText = fruit;
14    |   $li.addEventListener('click', alertFruitBuilder(fruit));
15    |   $ul.appendChild($li);
16  });
17
18  document.body.appendChild($ul);
19
```

# 클로저 활용 사례

- ▶ 콜백 함수 내부에서 외부 데이터를 사용하고자 할 때

```
1
2  const arr = [10, 20, 30, 40, 50];
3
4  for (var i = 0; i < arr.length; ++i) {
5      |   setTimeout(function() {
6      |       |   console.log('The index of this number is: ' + i);
7      |       |   }, 1000)
8      |   }
9  }
```

```
1
2  const arr = [10, 20, 30, 40, 50];
3
4  for (var i = 0; i < arr.length; ++i) {
5      |   setTimeout(function(index) {
6      |       |   return function() {
7      |       |       |   console.log('The index of this number is: ' + index);
8      |       |       |   }
9      |       |   }(i), 1000)
10     |   }
11 }
```

```
1
2  const arr = [10, 20, 30, 40, 50];
3  for (let i = 0; i < arr.length; ++i) {
4      |   setTimeout(function() {
5      |       |   console.log('The index of this number is: ' + i);
6      |       |   }, 1000)
7      |   }
8  }
```

# 클로저 활용 사례

## ▶ 접근 권한 제어 (정보 은닉)

- ▶ 내부 로직에 대해 외부로의 노출을 최소화해서 모듈 간의 결합도를 낮추고 유연성을 높이고자 하는 현대 프로그래밍의 중요한 개념 중 하나
- ▶ 자바스크립트는 변수 자체에 이러한 접근 권한을 부여하도록 설계되어 있지 않음
  - ▶ 클로저를 이용하면 함수 차원에서 public 한 값과 private 한 값 구분 가능
- ▶ 외부에 제공하고자 하는 정보들을 모아서 return 하고, 내부에서만 사용할 정보들을 return 하지 않음
  - ▶ return 한 변수들은 public member, 그렇지 않은 변수들은 private member

# 클로저 활용 사례

## ▶ 자동차 경주 게임 구현

- ▶ 각 턴마다 주사위를 굴려 나온 숫자(km) 만큼 이동한다.
- ▶ 차량 별로 연료량(fuel)과 연비(power)는 무작위로 생성된다.
- ▶ 남은 연료가 이동할 거리에 필요한 연료보다 부족하면 이동하지 못한다.
- ▶ 모든 유저가 이동할 수 없는 턴에 게임이 종료된다.
- ▶ 게임 종료 시점에 가장 멀리 이동해 있는 사람이 승리한다.



## 간단한 자동차 객체

```
1
2  var car = {
3      fuel: Math.ceil(Math.random() * 10 + 10), // 연료(L)
4      power: Math.ceil(Math.random() * 3 + 2), // 연비(km/L)
5      moved: 0, // 총 이동거리
6
7      run: function () {
8          var km = Math.ceil(Math.random() * 6);
9          var wasteFuel = km / this.power;
10         if (this.fuel < wasteFuel) {
11             console.log('이동불가');
12             return;
13         }
14         this.fuel -= wasteFuel;
15         this.moved += km;
16         console.log(km + 'km 이동 (총 ' + this.moved + 'km)');
17     },
18 };
19
```

## 클로저로 변수 를 보호한 자 동차 객체 (1)

```
1
2  var createCar = function () {
3      var fuel = Math.ceil(Math.random() * 10 + 10); // 연료(L)
4      var power = Math.ceil(Math.random() * 3 + 2); // 연비(km / L)
5      var moved = 0; // 총 이동거리
6
7      return {
8          get moved() {
9              return moved;
10         },
11         run: function () {
12             var km = Math.ceil(Math.random() * 6);
13             var wasteFuel = km / power;
14             if (fuel < wasteFuel) {
15                 console.log('이동불가');
16                 return;
17             }
18             fuel -= wasteFuel;
19             moved += km;
20             console.log(km + 'km 이동 (총 ' + moved + 'km). 남은 연료: ' + fuel);
21         },
22     };
23 };
24
25 var car = createCar();
26
```

## 클로저로 변수 를 보호한 자 동차 객체 (2)

```
1
2  var createCar = function () {
3      var fuel = Math.ceil(Math.random() * 10 + 10); // 연료(L)
4      var power = Math.ceil(Math.random() * 3 + 2); // 연비(km / L)
5      var moved = 0; // 총 이동거리
6
7      var publicMembers = {
8          get moved() {
9              return moved;
10         },
11         run: function () {
12             var km = Math.ceil(Math.random() * 6);
13             var wasteFuel = km / power;
14             if (fuel < wasteFuel) {
15                 console.log('이동불가');
16                 return;
17             }
18             fuel -= wasteFuel;
19             moved += km;
20             console.log(km + 'km 이동 (총 ' + moved + 'km). 남은 연료: ' + fuel);
21         },
22     };
23
24     Object.freeze(publicMembers);
25     return publicMembers;
26 };
27
28 var car = createCar();
29
```

# 클로저 활용 사례

## ▶ 클로저를 활용해 접근 권한 제어 방법

1. 함수에서 지역 변수 및 내부 함수 등을 생성
2. 외부에 접근 권한을 주고자 하는 대상 들로 구성된 참조형 데이터 (대상이 여럿일 때는 객체 또는 배열, 하나일 때는 함수)를 return

▶ return 한 변수들은 공개 멤버, 그렇지 않은 변수들은 비공개 멤버

# 클로저 활용 사례

## ■ 부분 적용 함수 (partially applied function)

- $n$ 개의 인자를 받는 함수에 미리  $m$ 개의 인자만 넘겨 기억 시켰다가, 나중에  $(n-m)$ 개의 인자를 넘기면 비로소 원래 함수의 실행 결과를 얻을 수 있게끔 하는 함수

bind 함수를 활용한  
부분 적용 함수

```
1
2  ✓ var add = function () {
3      var result = 0;
4      ✓ for (var i = 0; i < arguments.length; i++) {
5          result += arguments[i];
6      }
7      return result;
8  };
9
10 var addPartial = add.bind(null, 1, 2, 3, 4, 5);
11 console.log(addPartial(6, 7, 8, 9, 10));
12
```

# 클로저 활용 사례

## ■ 부분 적용 함수 (partially applied function)

■ this에 관여하지 않는 부분 적용 함수를 구현하려면?

```
1
2 var partial = function () {
3     var originalPartialArgs = arguments;
4     var func = originalPartialArgs[0];
5     if (typeof func !== 'function') {
6         throw new Error('첫 번째 인자가 함수가 아닙니다.');
```

7 }

```
8     return function () {
9         var partialArgs = Array.prototype.slice.call(originalPartialArgs, 1);
10        var restArgs = Array.prototype.slice.call(arguments);
11        return func.apply(this, partialArgs.concat(restArgs));
12    };
13 };
14
15 var add = function () {
16     var result = 0;
17     for (var i = 0; i < arguments.length; i++) {
18         result += arguments[i];
19     }
20     return result;
21 };
22
```

```
22
23 var addPartial = partial(add, 1, 2, 3, 4, 5);
24 console.log(addPartial(6, 7, 8, 9, 10));
25
26 var dog = {
27     name: '강아지',
28     greet: partial(function (prefix, suffix) {
29         return prefix + this.name + suffix;
30     }, '왹왹, '),
31 };
32
33 console.log(dog.greet('입니다!'));
34
```

```

1
2 var _ = Symbol.for('EMPTY_SPACE;')
3
4 var partial = function () {
5     var originalPartialArgs = arguments;
6     var func = originalPartialArgs[0];
7     if (typeof func !== 'function') {
8         throw new Error('첫 번째 인자가 함수가 아닙니다.');
```

인자들을 원하는 위치에 미리 넣어놓고 나중 빈 자리에 인자를 채워넣어 실행

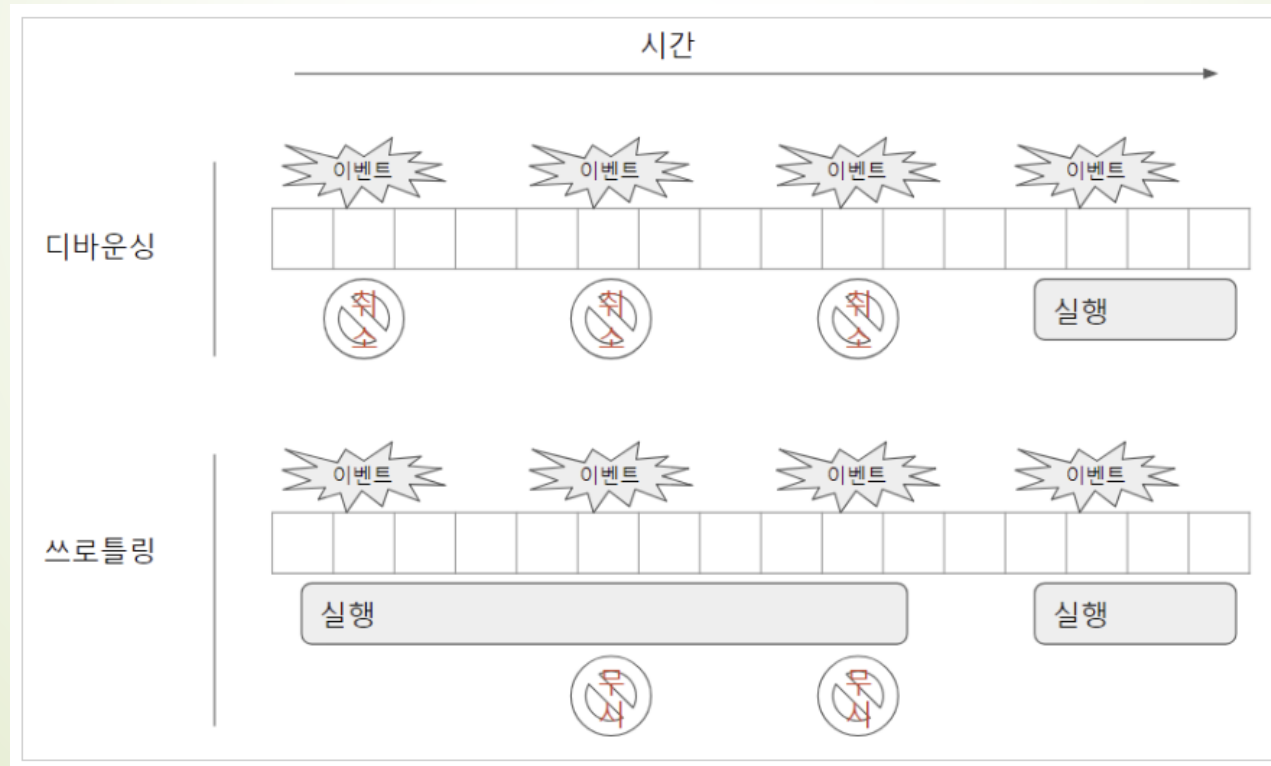
```

9     }
10    return function () {
11        var partialArgs = Array.prototype.slice.call(originalPartialArgs, 1);
12        var restArgs = Array.prototype.slice.call(arguments);
13        for (var i = 0; i < partialArgs.length; i++) {
14            if (partialArgs[i] === _) {
15                partialArgs[i] = restArgs.shift();
16            }
17        }
18        return func.apply(this, partialArgs.concat(restArgs));
19    };
20 };
21
22 var add = function () {
23     var result = 0;
24     for (var i = 0; i < arguments.length; i++) {
25         result += arguments[i];
26     }
27     return result;
28 };
29
29
30 var addPartial = partial(add, 1, 2, _, 4, 5, _, _, 8, 9);
31 console.log(addPartial(3, 6, 7, 10));
32
33 var dog = {
34     name: '강아지',
35     greet: partial(function (prefix, suffix) {
36         return prefix + this.name + suffix;
37     }, '왈왈, '),
38 };
39
40 console.log(dog.greet(' 배고파요!'));
41
```



# 클로저 활용 사례

- ▶ 부분 적용 함수 (partially applied function)의 예
  - ▶ 디바운싱 (debouncing)
    - ▶ 디바운싱은 이벤트가 빈번하게 발생할 때 마지막 이벤트만 실행하는 방법
  - ▶ 쓰로틀링 (throttling)
    - ▶ 쓰로틀링은 이벤트가 발생하여 실행 중일 때, 실행된 이벤트가 끝나기전에 발생하는 이벤트는 무시하는 방법



# 클로저 활용 사례

```
1
2  var timer;
3  document.querySelector('#input').addEventListener('input', function (e) {
4      if (timer) {
5          clearTimeout(timer);
6      }
7      timer = setTimeout(function () {
8          console.log('event 발생', e.target.value);
9      }, 200);
10 });
11
```

```
1
2  var timer;
3  document.querySelector('#input').addEventListener('input', function (e) {
4      if (!timer) {
5          timer = setTimeout(function () {
6              console.log('event 발생', e.target.value);
7              timer = null;
8          }, 200);
9      }
10 });
11
```

# 클로저 활용 사례

## ■ 커링 함수 (currying function)

- 여러 개의 인자를 받는 함수를 하나의 인자만 받는 함수로 나눠서 순차적으로 호출될 수 있게 체인 형태로 구성한 것

```
1
2  var curry3 = function (func) {
3      |   return function (a) {
4          |       return function (b) {
5              |           return func(a, b);
6              |       };
7          |   };
8      };
9
10  var getMaxWith10 = curry3(Math.max)(10);
11  console.log(getMaxWith10(8));
12  console.log(getMaxWith10(25));
13
14  var getMinWith10 = curry3(Math.min)(10);
15  console.log(getMinWith10(8));
16  console.log(getMinWith10(25));
```

# 클로저 활용 사례

## ▶ 커링 함수 (currying function)

```
1
2  var curry5 = function (func) {
3      return function (a) {
4          return function (b) {
5              return function (c) {
6                  return function (d) {
7                      return function (e) {
8                          return func(a, b, c, d, e);
9                      };
10                 };
11             };
12         };
13     };
14 };
15
16 var getMax = curry5(Math.max);
17 console.log(getMax(1)(2)(3)(4)(5));
18
```

```
1
2  var curry5 = func => a => b => c => d => e => func(a, b, c, d, e);
3
4  var getMax = curry5(Math.max);
5  console.log(getMax(1)(2)(3)(4)(5));
6
7
```



# 클로저 활용 사례

- 커링 함수 (currying function)
  - 마지막 인자가 전달될 때까지 함수 실행이 지연됨
  - 함수형 프로그래밍 패러다임에서는 이를 지연 실행 (lazy execution)이라고 함

```
1
2  var getInformation = baseUrl => path => id => fetch(baseUrl + path + '/' + id);
3
4  var imageUrl = 'http://imageAddress.com/';
5  var productUrl = 'http://productAddress.com/';
6
7  // 이미지 타입별 요청 함수 준비
8  var getImage = getInformation(imageUrl);    // http://imageAddress.com/
9  var getEmoticon = getImage('emoticon');    // http://imageAddress.com/emoticon
10 var getIcon = getImage('icon');             // http://imageAddress.com/icon
11
12 // 제품 타입별 요청 함수 준비
13 var getProduct = getInformation(productUrl); // http://productAddress.com/
14 var getFruit = getProduct('fruit');         // http://productAddress.com/fruit
15 var getVegetable = getProduct('vegetable'); // http://productAddress.com/vegetable
16
17 var emoticon1 = getEmoticon(100);           // http://imageAddress.com/emoticon/100
18 var emoticon2 = getEmoticon(102);           // http://imageAddress.com/emoticon/102
19 var icon1 = getIcon(205);                   // http://imageAddress.com/icon/205
20 var icon2 = getIcon(234);                   // http://imageAddress.com/icon/234
21 var fruit1 = getFruit(300);                  // http://productAddress.com/fruit/300
22 var fruit2 = getFruit(400);                  // http://productAddress.com/fruit/400
23 var vegetable1 = getVegetable(456);         // http://productAddress.com/vegetable/456
24 var vegetable2 = getVegetable(789);         // http://productAddress.com/vegetable/789
25
```



# 정리

- 클로저란 어떤 함수에서 선언한 변수를 참조하는 내부 함수를 외부로 전달할 경우, 함수의 실행 컨텍스트가 종료된 후에도 해당 변수가 사라지지 않는 현상
- 내부 함수를 외부로 전달하는 방법에는 함수를 return 하는 경우와 콜백으로 전달하는 경우가 있음
- 클로저는 메모리를 계속 차지하게 되므로 더는 사용하지 않게 된 클로저에 대해서는 메모리를 차지하지 않도록 관리해주어야 함



## 복습 문제

```
1
2  function outer() {
3      var a = 0;
4
5      return () => {
6          ++a;
7      };
8  }
9
10 var f1 = outer();
11 var f2 = outer();
12
13 console.log(f1());
14 console.log(f1());
15 console.log(f2());
16 console.log(f2());
17
```